BÀI TẬP THỰC HÀNH
THU THẬP DỮ LIỆU WEB DÙNG PYTHON

1. Mục tiêu thực hành

- Hiểu web scraping là gì, vì sao được sử dụng và các giới hạn cần lưu ý.
- Hiểu cơ chế HTTP Request/Response và vai trò của HTML/CSS trong web scraping.
- Sử dụng Requests để tải nội dung web và BeautifulSoup để parse HTML.
- Trích xuất dữ liệu từ một trang và nhiều trang, sau đó lưu thành Pandas DataFrame/CSV.
- Đọc dữ liệu XML và JSON, chuyển dữ liệu thành dạng có cấu trúc.
- Hiểu XPath và sử dụng Selenium cho website có nội dung động hoặc cần tương tác.
- Biết tổ chức một crawler nhỏ có delay, timeout, xử lý lỗi và giới hạn số trang.
- Tuân thủ Terms of Use, robots.txt, quyền riêng tư và nguyên tắc không gây tải quá mức cho website.

2. Chuẩn bị

- Python 3.x.
- Jupyter Notebook/JupyterLab hoặc VS Code.
- Thư mục data đi kèm.
- Kết nối Internet cho các ví dụ live.
- Chrome/Chromium nếu thực hành Selenium.
- Cài đặt thư viện python:

```
pip install requests beautifulsoup4 pandas lxml selenium
```

3. Nội dung thực hành

1. Giới thiệu Web Scraping

Web scraping là quá trình tự động tải, phân tích và tổ chức dữ liệu lấy từ web. Trong bài thực hành này, mục tiêu không phải thu thập càng nhiều càng tốt mà là xác định đúng dữ liệu cần cho bài toán rồi chọn công cụ phù hợp.

URL -> HTTP Request -> HTML/XML/JSON -> Parser -> DataFrame -> CSV/Phân tích

2. HTTP Request/Response

Khi trình duyệt mở một URL, client gửi HTTP request đến web server và nhận response. Với web scraping, thư viện Requests thực hiện vai trò tương tự ở mức HTTP.

```
import requests

url = "https://example.com"
response = requests.get(url, timeout=10)

print("Status:", response.status_code)
print("Content-Type:", response.headers.get("Content-Type"))
print(response.text[:300])
```

2.1. Một số status code

Mã	Ý nghĩa
200	Thành công
301/302	Redirect

403	Bị từ chối
404	Không tìm thấy
429	Quá nhiều request
500	Lỗi server

2.2. Hàm tải trang có xử lý lỗi

```
def fetch_page(url, timeout=10):
    try:
        r = requests.get(
            url,
            timeout=timeout,
            headers={"User-Agent": "Educational-Web-Scraping-Lab/1.0"}
        )
        r.raise_for_status()
        return r
    except requests.RequestException as e:
        print("Request error:", e)
        return None
```

3. HTML và CSS

HTML cấu trúc nội dung trang web; CSS định dạng cách hiển thị. Khi scrape, cần quan tâm đến tag, attribute, class và id.

- <h1>...<h6>: tiêu đề
- <p>: đoạn văn
- : nhóm phần tử
- <a href='...'>: liên kết
- <table>, <tr>, <th>, <td>: bảng
- <img src='...'>: hình ảnh
- class: có thể xuất hiện nhiều lần
- id: thường định danh phần tử duy nhất

```
html_sample = '''
<div class="product">
  <a href="/p1">Laptop A</a>
  <span class="price">1200</span>
</div>
'''
```

4. Pháp lý và đạo đức

Trước khi thu thập dữ liệu, cần kiểm tra điều khoản sử dụng, robots.txt và mục đích sử dụng dữ liệu.

- Ưu tiên dữ liệu công khai và API chính thức nếu có.
- Không vượt qua login, CAPTCHA hoặc cơ chế bảo vệ.
- Không gửi request quá nhanh.
- Hạn chế số trang/request theo đúng nhu cầu.
- Không thu thập hoặc phổ biến dữ liệu cá nhân/nhạy cảm không cần thiết.
- Ghi rõ nguồn và thời điểm thu thập.

Lưu ý: Web scraping không có một giải pháp áp dụng cho mọi website. Cần kiểm tra cấu trúc trang, thử nghiệm và điều chỉnh.

5. Requests

Requests là thư viện chính để gửi HTTP request tới trang web hoặc API.

```
headers = {
   "User-Agent": "Mozilla/5.0 Educational Lab",
   "Accept-Language": "en-US,en;q=0.5"
}

r = requests.get(
   "https://example.com",
   headers=headers,
   timeout=10
)
print(r.status_code)
```

5.1. Query parameters

```
params = {"year": 2008}
r = requests.get("https://httpbin.org/get", params=params, timeout=10)
print(r.url)
```

6. BeautifulSoup

BeautifulSoup chuyển HTML thành cây đối tượng để tìm kiếm phần tử.

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html_sample, "html.parser")

print(soup.find("a").get_text(strip=True))
print(soup.find("a").get("href"))
print(soup.find("span", class_="price").get_text(strip=True))
```

6.1. find_all và CSS selector

```
for p in soup.select("div.product"):
   name = p.select_one("a").get_text(strip=True)
   price = p.select_one(".price").get_text(strip=True)
   print(name, price)
```

7. Scraping một trang

Code đính kèm có ví dụ CityU News: tải một trang, tìm bảng chứa danh sách tin, lấy title, URL và date rồi đưa vào DataFrame.

```
import pandas as pd

cityu_html = '''
<table class="cityu-width-full">
  <tr>
    <td><a href="news1.html">Research Seminar 2008</a></td>
    <td><div>News</div><div>2008-01-15</div></td>
  </tr>
  <tr>
    <td><a href="news2.html">Student Award</a></td>
    <td><div>News</div><div>2008-03-22</div></td>
  </tr>
```

```
</table>
'''

page = BeautifulSoup(cityu_html, "html.parser")
container = page.find("table", class_="cityu-width-full")

records = []
for row in container.find_all("tr"):
    a = row.find("a")
    divs = row.find_all("div")
    if a:
        records.append({
            "title": a.get_text(strip=True),
            "url": a.get("href"),
            "date": divs[-1].get_text(strip=True) if divs else None
        })

df = pd.DataFrame(records)
df
```

Lưu ý: Các website trong code gốc có thể đã thay đổi cấu trúc. Khi đó cần inspect HTML hiện tại và cập nhật selector, không nên cố giữ selector cũ.

8. Scraping nhiều trang

Với nhiều trang/nhiều năm, nên đóng gói logic scraping thành hàm và lặp theo danh sách URL hoặc năm.

```
from time import sleep
from random import randint

def scrape_year(year):
    url = f"http://www.cityu.edu.hk/com/News_News.aspx?year={year}"

    try:
        r = requests.get(url, timeout=15)
        r.raise_for_status()
    except requests.RequestException as e:
        print(year, e)
        return pd.DataFrame()

    sleep(randint(1, 3))
    soup = BeautifulSoup(r.content, "html.parser")
    # parse tiếp tại đây...
    return pd.DataFrame()
```

8.1. Các nguyên tắc

- Đặt delay giữa request.
- Không dùng except: trống.
- Ghi log URL lỗi.
- Đặt timeout.
- Lưu source URL và thời điểm thu thập.
- Khi chỉ cần một số trang, đặt giới hạn rõ ràng.

9. Xây dựng crawler nhỏ

Crawler là bước mở rộng của scraping nhiều trang. Nên tách chức năng thành fetch, parse, extract và save.

```
from urllib.parse import urljoin, urlparse
from collections import deque
import time

def fetch_html(url, delay=1):
    time.sleep(delay)
    try:
        r = requests.get(url, timeout=15)
        r.raise_for_status()
        return r.text
    except requests.RequestException:
        return None

def same_domain(url1, url2):
    return urlparse(url1).netloc == urlparse(url2).netloc
```

9.1. Giới hạn crawler

- max_pages
- visited URL
- same-domain constraint
- delay/rate limit
- timeout
- error logging

10. XML

XML có cấu trúc cây và thường được sử dụng để trao đổi dữ liệu có cấu trúc.

```
import xml.etree.ElementTree as ET

xml_text = '''
<person>
  <name>Chuck</name>
  <phone type="intl">+1 734 303 4456</phone>
  <email hide="yes"/>
</person>
'''

root = ET.fromstring(xml_text)
print(root.find("name").text)
print(root.find("email").get("hide"))
```

10.1. Đọc file XML LegCo

```
tree = ET.parse("data/cm_vote_20181010.xml")
root = tree.getroot()

records = []
for member in root.iter("member"):
    vote = member.find("vote")
    records.append({
```

```
        "name": member.get("name-ch"),
        "vote": vote.text if vote is not None else None
    })

vote_df = pd.DataFrame(records)
vote_df.head()
```

11. JSON

JSON thường ánh xạ trực tiếp sang dict/list của Python.

```
import json

json_text = '''
{
  "name": "Chuck",
  "phone": {
    "type": "intl",
    "number": "+1 734 303 4456"
  }
}
'''

info = json.loads(json_text)
print(info["name"])
print(info["phone"]["number"])
```

11.1. JSON nested sang DataFrame

```
nested = [
    {
        "created_at": "2026-10-01",
        "text": "Data analytics is useful",
        "user": {
            "screen_name": "user_a",
            "followers_count": 120
        }
    }
]

json_df = pd.json_normalize(nested)
json_df
```

12. XPath

XPath dùng để xác định phần tử trong cây HTML/XML. Có thể dùng với lxml hoặc Selenium.

```
from lxml import html

tree = html.fromstring('''
<div class="product">
  <a>Laptop A</a>
  <span class="price">1200</span>
</div>
''')

print(tree.xpath('//div[@class="product"]/a/text()'))
print(tree.xpath('//span[@class="price"]/text()'))
```

12.1. CSS selector hay XPath?

- CSS selector thường ngắn và dễ đọc khi cấu trúc class/id rõ ràng.
- XPath linh hoạt hơn khi cần điều hướng quan hệ cha-con hoặc tìm theo điều kiện phức tạp.
- Ưu tiên selector ổn định, tránh selector phụ thuộc vị trí tuyệt đối nếu không cần thiết.

13. Selenium

Selenium dùng khi trang cần JavaScript, click, search, scroll hoặc tương tác trình duyệt.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--headless=new")

# driver = webdriver.Chrome(options=options)
# driver.get("https://example.com")
# title = driver.find_element(By.TAG_NAME, "h1").text
# print(title)
# driver.quit()
```

13.1. Cập nhật cú pháp Selenium

Code cũ thường dùng find_elements_by_xpath(). Selenium hiện đại dùng By.XPATH.

```
# Cũ:
# driver.find_elements_by_xpath("//div")

# Mới:
elements = driver.find_elements(By.XPATH, "//div")
```

Lưu ý: Các website trong code gốc có thể đã thay đổi cấu trúc. Khi đó cần inspect HTML hiện tại và cập nhật selector, không nên cố giữ selector cũ.

BÀI TẬP MỞ RỘNG

Web Data Collection Pipeline

Xây dựng pipeline thu thập dữ liệu tin/bài viết công khai.

Output đề nghị

Trường	Ý nghĩa
title	Tiêu đề
date	Ngày
url	Đường dẫn
content	Nội dung
source	Nguồn
collected_at	Thời điểm thu thập
status	Trạng thái

Các bước

- Chọn nguồn dữ liệu công khai.
- Kiểm tra Terms of Use/robots/API.
- Inspect HTML và xác định selector.
- Viết fetch_page().
- Viết parser danh sách.

- Viết parser trang chi tiết.
- Đặt delay và timeout.

Xử lý lỗi.

Chuẩn hóa dữ liệu.

Lưu CSV.

Hàm chuẩn hóa record

```
from datetime import datetime

def normalize_record(record):
    return {
        "title": (record.get("title") or "").strip(),
        "date": record.get("date"),
        "url": record.get("url"),
        "content": (record.get("content") or "").strip(),
        "source": record.get("source"),
        "collected_at": datetime.now().isoformat(timespec="seconds"),
        "status": record.get("status", "ok")
    }
```

4. Bài tập về nhà

Bài 1: HTML

- Lấy tên, giá và link từ danh sách sản phẩm HTML.
- Lưu vào DataFrame.

Bài 2: Requests + BeautifulSoup

- Chọn trang web công khai, tĩnh.
- GET request, kiểm tra status, trích ít nhất 3 trường, lưu CSV.

Bài 3: Multi-page

- Crawl tối đa 5 trang.
- Delay ít nhất 1 giây.
- Không crawl URL trùng.
- Lưu URL lỗi.

Bài 4: XML

- Đọc cm_vote_20181010.xml.
- Trích tên thành viên và phiếu.
- Đếm số phiếu từng loại.

Bài 5: JSON

- Tạo JSON nested user + posts.
- json.loads().
- pd.json_normalize().

Bài 6: XPath

- Viết XPath lấy product, name, price.
- So sánh với CSS selector.

Bài 7: Selenium

- Mở một website demo hợp pháp.
- Nhập từ khóa, search, lấy 5 kết quả, đóng driver.

5. Câu hỏi ôn tập

Câu hỏi ôn tập

1. Web scraping khác API ở điểm nào?
2. Vì sao cần kiểm tra status code trước khi parse?
3. class và id trong HTML khác nhau thế nào?
4. Khi nào nên dùng Requests + BeautifulSoup?
5. Khi nào cần Selenium?
6. Tại sao cần delay giữa các request?
7. XML và JSON khác nhau như thế nào về cấu trúc?
8. XPath khác CSS selector ở điểm nào?
9. Vì sao không nên dùng except: trống?
10. Crawler cần những giới hạn nào để tránh chạy không kiểm soát?

Yêu cầu nộp bài

- Notebook đã chạy và có output ở các phần được yêu cầu.
- File CSV/XML/JSON kết quả nếu có.
- Bài tập có mô tả nguồn dữ liệu và selector sử dụng.
- Ghi rõ website/API đã dùng và ngày thu thập.
- Có nhận xét ngắn về lỗi gặp phải và cách xử lý.
- Không nộp dữ liệu được thu thập bằng cách vượt qua cơ chế bảo vệ của website.

Tổng kết

HTTP -> HTML/XML/JSON -> Requests -> Parser -> Crawler -> Selenium -> DataFrame

Cần nhớ: Trước hết xác định đúng dữ liệu cần thu thập, sau đó chọn phương pháp đơn giản nhất có thể. Nếu HTML tĩnh, ưu tiên Requests/BeautifulSoup; nếu là API, ưu tiên API; chỉ dùng Selenium khi cần nội dung động hoặc tương tác trình duyệt.